Design Report: How I Built Doodlebot – The Lost Kingdom

Tala Alothaim · Technical exam · An autonomous navigation game and a 30-minute Grade 5 lesson

The one-page reflection summarises what I made. This report walks through how I built it, step by step, and what made me change direction along the way.

Step 1. Understanding the Task, the Audience and the Features

I started by reading the technical requirements carefully, to understand both the task and its purpose: helping children understand how a robot can find its way through a world it cannot see. I listed every feature the game needed: a maze editor, the real map and Doodlebot's own map, a sensor range setting, exploration, path planning, live numbers, a character with emotions and a voice chat. Then I thought about who would use it. Ten-year-olds need an interface that is simple, clear and straightforward, characters they can relate to, and ideas presented one step at a time.

Step 2. Fitting Many Features on One Screen

That list raised my main design question: how can I present this many features on one dashboard without it feeling overwhelming or confusing for a child? My answer was to organise the game into three stages shown at the top of the page, Build → Explore → Results, and to show only what a child needs at each stage. Each stage has one main button, and only two numbers are shown live while exploring (steps to finish mapping and cells discovered). The full results only appear at the end, on a results card.

Step 3. Testing and Changing the Interface

I tested the interface many times, and each time I went back and changed things that did not feel right for a child:

- **Sizes:** I made the buttons larger (at least 48 px tall) and adjusted the size of the chat window so it no longer covered the maze.
- **Drop-down menus:** I turned several features into drop-down sections (Build Tools, Mission Settings and My Experiments) that stay closed until needed and open independently. The second algorithm (A*) sits inside its own drop-down, so it does not distract from the main experiment.
- **One map at a time:** my first layout showed the real map and Doodlebot's map side by side. As an adult, I still found it confusing to have two maps in front of me, so I knew a child would too. I changed it to one large map that matches the stage (the real world while building, Doodlebot's map while exploring), with a toggle to show both when a child wants to compare them.

Step 4. Testing with a Real User

Even after the interface was finished, I felt the front end was still not clear enough, so I brought my sister to test it. Without explaining anything, I asked her what she understood at first glance. Her notes were that she understood the game and its goal, but had slight difficulty understanding the steps and what Doodlebot is doing while exploring, meaning what actually happens when the student presses Start Exploring.

That feedback gave me the idea of a home page with a tutorial of the steps before the child starts exploring. I built a welcome page with the castle story and a four-step animated tutorial, "How Doodlebot Thinks": look around, explore the unknown, think and plan, and rescue (or try again if the path is blocked). The tutorial replays frames recorded from the real algorithm on small practice mazes, so it never shows anything the game does not do. Students can skip it, and the Home button pauses the game without losing their progress. When I reviewed it, the explanations disappeared too fast for slow readers, so I made them stack up instead.

Step 5. Designing the Characters

I wanted Doodlebot to have specific traits and a look that children could connect with. I chose a simple robot illustration as a style reference ("Cute robot rocking out" on Magnific, found through Google) and used ChatGPT to generate the same character in different emotions. I then created a table of where each emotion should be used, so that every expression is tied to a real moment in the game:

Game moment	Emotion	How children see it
Building the maze	Idle	Calm smile, slow blink
Exploring before the map is complete	Curious	Eyes look where it drives, "?" on its chest
Princess-Bot enters the sensor range	Excited	Star eyes, jumping, sparkles
Planning the shortest path	Thinking	Hand on chin, loading dots
Shortest path found	Happy	Big smile, waving, heart
No path exists	Sad	Tears, broken heart, asks for help

The colours follow the story. Doodlebot is the boy of the story, like a little prince in blue, setting out to find and rescue the princess, so Princess-Bot is pink. Blue and pink also stand out clearly from each other and from the maze. Princess-Bot also has a different shape (a tall, floating, crowned egg), not just a different colour, so children can tell the two robots apart at a glance, even on a small maze cell. Finally, I rebuilt Doodlebot as one layered drawing so it can blink, look where it drives and move its mouth only while it speaks, and each emotion is shown on its face, chest screen and antenna, so children never rely on colour alone.

Step 6. Building the Navigation and the Chat

Doodlebot starts knowing only the grid size and its own position. Every step, it senses every cell within its sensor range and adds it to its own map. To explore, it searches its own map with breadth-first search (BFS) and moves one step toward the nearest known free cell that touches an unknown cell, so "nearest" means by path, not straight-line distance. Exploration stops only when no reachable unknown cell is left. Doodlebot then returns to its start and plans the shortest path on its finished map (BFS, or A* as an extra), never on the real maze. Ties are always broken in the same order, so the same maze always gives the same run, which keeps the classroom experiment fair. While building it, I hit a few problems:

- **Exploring after finding Princess-Bot.** When she was walled in, Doodlebot kept exploring after sensing her, which confused me at first. I kept the rule, because an unexplored square could hide a gap, and made Doodlebot explain this instead.
- **The chatbot refusing questions.** It answered "That's outside my maze!" to almost everything, because speech recognition misheard "Doodlebot" and "maze". I added example answers to the prompt (kept in its own file) and corrected common mishearings.
- **Voice quality versus speed.** A natural OpenAI voice sounds friendlier but adds about a second of delay, so the browser voice stays as an automatic backup, and the chat still answers offline without an API key.

Step 7. Checking the Game

I wrote 11 automated tests for the parts that are easy to get subtly wrong (choosing the nearest frontier by path distance, stopping correctly, true shortest paths on 60 random mazes, and odd cases such as an enclosed start or a walled-in Princess-Bot). I also played every edge case myself, tested the voice chat in Chrome and checked the layout on desktop, tablet and phone. Checking against the requirements, I noticed I had completed several optional items without planning to, because they made the game feel complete: random solvable mazes, an animated planner search, A*, a results panel, all emotions animated, the story and tutorial and, for the lesson, a student worksheet..

Step 8. Designing the Lesson

Since the lesson is only 30 minutes, my goal was for students to understand how autonomous navigation starts and works: a robot senses, builds its own map, and only then plans. I built the lesson around one experiment instead of every game feature, so this idea settles, and kept BFS versus A* as an extension.

Segment	What happens	Why
Warm-up (6 min)	Mystery Map: the class acts as Doodlebot's brain and can only see nearby squares	Children feel the problem before seeing the solution
Concept and gameplay (18 min)	Tutorial, prediction, then the same maze with sensor range 1 and then 4, in pairs	A fair test with one change; pairs finish faster and discuss more
Reflection (6 min)	Three reflection questions and a voice question to Doodlebot	Students explain the result in their own words

Most children expect a bigger sensor to give a shorter path, but the results show **78 versus 29 steps to finish mapping, while the shortest path stays 21**. Predicting first, recording real numbers and explaining the difference is the evidence of learning, captured on the worksheet and judged with a simple rubric. The lesson plan guides the facilitator minute by minute, with sample sentences, a check-for-understanding question, expected answers and troubleshooting.

Step 9. Consistency, AI Tools and Next Steps

Last, I went over the code, lesson plan, slides and worksheet together to make sure they use the same maze (ID 1X6Y), the same numbers, the same steps and the same words everywhere.

- **Image source and ChatGPT:** the "Cute robot rocking out" vector illustration on Magnific (Freepik), found through Google, was my style reference; ChatGPT generated the emotion variations.
- **Claude:** my coding partner. I described my plans, design and decisions, and it helped implement and refine the code and materials. I checked its work with the tests, by playing every case, and by questioning anything that did not feel right.

Final notes

- With more time, I would test the game with more children and add a tele-operated mode, so they can drive Doodlebot themselves and compare their steps with the robot's.
- I built the game and the lesson materials in about 9 hours of the 24-hour window because of university exams in my schedule, so I chose to focus on the required items and the most useful optional ones.
- What I enjoyed most was thinking about my audience first, and letting real testing, even with my own sister, shape the game, the lesson and the way the ideas are presented.